

Name: Rhilo Sotto
RED ID: 130551574
Date: 9/27/2024

Lab 3 Report

1. What does your program do?

The program iteratively scans a GPIO keypad and transmits the button pressed, with one button press transmits one character. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost. The signal is viewed through the built-in data visualizer.

2. What variables/registers does your program use?

The program uses the **UBRR0H** and **UBRR0L** registers to set the BAUD RATE using the UBBR equation (high stores 8 high bits, and low stores 8 low bits) which is calculated beforehand.

The **UCSR0B** register enables the receiver (RXEN0 - pin 3 = 1) and transmitter (TXEN0 - pin 4 = 1) of the microcontroller (only the transmitter is used in this lab).

The **UCSR0C** register to set the microcontroller to Asynchronous USART mode (UMSEL00 and UMSEL01 - pins 6 and 7 = 0), the signal having 8 data bits (UCSZ00 and UCSZ01 - pins 1 and 2 = 1), to disable the parity bit (UPM00 and UPM01 - pins 4 and 5 = 0), and signal only sending 1 stop bit (USBS0 - pin 3 = 0). The signal is 10 bits total including the start bit.

The **UCSR0A** register is used to check whether the transmit buffer is empty (UDRE0 - pin 5 = 1).

The **UDR0** register is used to transmit the data (8 bit chars).

The **DDRD** register is used to set the rows of the GPIO to outputs

The **DDRB** register is used to set the columns of the GPIO to inputs

The **PORTD** register is used to set rows to logic high and isolate the row of a button pressed during iterative scanning.

The **PORTB** register is used to enable the pull-up resistor property and make the default input to the port logic high, and logic low when the button is pressed.

The **PINB** register is used to read the Port B input pins during iterative scanning.

The 2D char array **keychar** is used to map the indices **i,j** from the main program's double for loop to characters corresponding to the GPIO.

3. What is the main algorithm/logic of your program?

The main program calls the `usart_init` function to initialize all of the necessary microcontroller settings, including setting the baud rate, enabling the receiver and transmitter, setting the USART mode to asynchronous, an 8 bit data frame, disabling the parity bit, and only sending 1 stop bit.

Then the program initializes the necessary GPIO settings for Port B and Port D, where `PORTD` and `DDRD` correspond to output rows, and `PORTB` and `DDRB` correspond to input columns with pull-up property enabled.

The program enters the infinite while loop, where it will check whether a button on the GPIO has been pressed by iteratively scanning each row and their columns. A double for loop is used to scan through each row (index `i`, 4-7), and in that row, each column (index `j`, 0-3). The row being scanned is set to logic low (setting its pin to 0), and then each column is checked for whether the pin is logic low. If so, then the corresponding indices `i`, `j` are mapped to a 2D char array and that char is transmitted through USART. The program enters a busy while loop, which will wait until the button pressed is released before continuing the iterative scan. After each column is read, the row is set back to logic high before repeating the row loop. This will continue indefinitely, scanning from the top left to the bottom right for button inputs.

4. What external hardware connections did you make?

The microcontroller is the transmitter which is hooked up via USB to my computer and the output is read through the data visualizer in COM3. The GPIO is connected to my microcontroller by pins `PD4-PD7`, and `PB0-PB3`, where the rows correspond to port D and columns correspond to port B.

Appendix (Code)

```
/*
 * Lab3.c
 *
 * Created: 9/19/2024 3:38:44 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock speed

#define BAUD 9600 // bits/sec
#define BAUDRATE ((F_CPU)/(BAUD*16UL) - 1)
```

```
#include <avr/io.h>
```

```
void usart_init(void) { // initialize AVR settings
    //UBRR0H &= 0; // clear high bits,
    UBRR0H = (BAUDRATE>>8); // shift register right by 8 bits
    UBRR0L = BAUDRATE; // set baud rate

    UCSR0B |= (1<<RXEN0) | (1<<TXEN0); // enable receiver and transmitter
    // only transmitter is used for this lab

    UCSR0C &= ~(1<<UMSEL00) | ~(1<<UMSEL01); // Asynchronous USART mode
    UCSR0C |= (1<<UCSZ00) | (1<<UCSZ01); // 8 data bit frame
    UCSR0C &= ~(1<<UPM01) | ~(1<<UPM00); // disable parity bit
    UCSR0C &= ~(1<<USBS0); // 1 stop bit
}
```

```
void usart_transmit(char data) { // transmit one char
    while ( !(UCSR0A & (1<<UDRE0)) ); // wait until transmit buffer is empty
    UDR0 = data; // write data to register
}
```

```
char usart_receive(void) { // receive one char
    while ( !(UCSR0A & (1<<RXC0)) ); // wait while data is received
    return UDR0; // return data
}
```

```
int main(void) {

    // map row/column to key
    char keychar[4][4] =
    {
        {'1', '2', '3', 'A'},
        {'4', '5', '6', 'B'},
        {'7', '8', '9', 'C'},
        {'*', '0', '#', 'D'}
    };

    /* initialization */
    usart_init();
    // Rows = PORTD, Columns = PORTB
}
```

```

// set rows to output
DDRD |= (1<<DDD4 | 1<<DDD5 | 1<<DDD6 | 1<<DDD7); // Port D (4-7)
// set column to input
DDRB &= ~(1<<DDB0 | 1<<DDB1 | 1<<DDB2 | 1<<DDB3); // Port B (0-3)
// pull up property enabled (columns)
PORTB |= (1<<PORTB0 | 1<<PORTB1 | 1<<PORTB2 | 1<<PORTB3);
// rows to logic high
PORTD |= (1<<PORTD4 | 1<<PORTD5 | 1<<PORTD6 | 1<<PORTD7);

while (1) {
    /* iterative scan */
    for (char i = 4; i < 8; ++i) {
        // set row[i] low
        PORTD &= ~(1<<i);
        for (char j = 0; j < 4; ++j) {
            // check column[j] if low
            if ( !(PINB & (1<<j)) ) { // first transition low (on)
                usart_transmit(keychar[i-4][j]);
                while ( !(PINB & (1<<j)) ) {} // wait until button is released
            }
        }
        // set row[i] high
        PORTD |= (1<<i);
    }
}
}

```